Mini Project on

CloudExecX – Auto-Scaling Code Execution Platform

Submitted in partial fulfillment of the requirements of the degree of

Bachelor of Engineering in Computer Science and Engineering (AI & ML) Department

Submitted by

Rishi Harishchandra Chaurasia

Under the guidance of:

Prof. Sona Bhoir

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)
VIVA INSTITUTE OF TECHNOLOGY

At Shirgoan, Virar (East), Tal. Vasai, Dist. Palghar – 401305

2025-26

Table of Contents

Abstract

CloudExecX is a cloud-native code execution platform designed to provide secure, scalable, and isolated environments for running user-submitted code. The platform enables users to write and execute programs in multiple programming languages while ensuring strong security and resource control through container-based sandboxing. Each execution request is processed through a queue-based architecture that decouples the application layer from the execution environment, allowing efficient workload distribution and simulated auto-scaling of worker processes.

The system architecture consists of a modern web interface built with Next.js, an API layer developed using Express.js and TypeScript, and a background worker system powered by BullMQ. Code execution tasks are queued using Redis and processed by worker services that spawn isolated Docker containers to safely run the submitted code. Strict security policies, including network isolation, limited CPU and memory allocation, execution timeouts, read-only filesystems, and non-root container execution, are enforced to prevent malicious activities and ensure fair resource usage.

CloudExecX supports multiple programming languages such as Python, JavaScript, and C++, and provides real-time execution output to the user. The platform is designed to operate entirely on free-tier cloud services, demonstrating how scalable cloud architectures can be implemented with minimal infrastructure cost. By combining containerization, queue-based job processing, and cloud deployment strategies, CloudExecX serves as a practical implementation of distributed system principles and modern cloud computing practices.

Introduction

With the rapid growth of cloud computing and online development platforms, the need for secure and scalable systems that can execute user-submitted code has significantly increased. Platforms such as online coding judges, educational tools, and remote development environments require reliable mechanisms to run code submitted by multiple users while ensuring system stability, security, and efficient resource utilization. However, executing untrusted code directly on a server poses serious risks, including system compromise, resource exhaustion, and unauthorized access.

CloudExecX is designed to address these challenges by providing a secure, cloud-native code execution platform that runs user programs inside isolated container environments. By leveraging containerization technology, the platform ensures that every execution task is sandboxed with strict resource limitations and security controls. This prevents malicious or poorly written code from affecting the host system or other users.

The platform follows a modern distributed architecture that separates the user interface, API layer, task queue, and execution workers. Users interact with a web-based interface where they can write and submit code. The request is processed by a backend API and placed into a queue using Redis. Worker processes then retrieve tasks from the queue and execute the code inside Docker containers with restricted permissions, limited CPU and memory usage, and controlled execution time.

CloudExecX supports multiple programming languages including Python, JavaScript, and C++. It also provides real-time output feedback to enhance the user experience. The system is designed to demonstrate how scalable and secure cloud-based execution platforms can be built using modern technologies such as Next.js, Express.js, BullMQ, Docker, and MongoDB.

Additionally, the platform is deployed entirely using free-tier cloud services, highlighting how distributed systems and scalable architectures can be implemented with minimal infrastructure cost. By combining queue-based task processing, containerized execution environments, and modern cloud deployment strategies, CloudExecX serves as a practical example of secure code execution systems and distributed cloud architecture.

Features

CloudExecX provides a modern, secure, and scalable environment for executing user-submitted code. The platform integrates cloud-native technologies and container-based sandboxing to ensure reliable performance and strong security. The key features of CloudExecX are described below.

1. Secure Code Execution

CloudExecX executes all user-submitted code inside isolated Docker containers. Each container runs with restricted permissions and resource limits, ensuring that the code cannot affect the host system or other users. Security measures such as network isolation, non-root execution, and restricted filesystem access protect the system from malicious activities.

2. Multi-Language Support

The platform supports multiple programming languages, allowing users to write and execute code in different environments. Currently supported languages include:

- Python (CPython 3.11)
- JavaScript (Node.js 20)
- C++ (GCC 13 with C++17 standard)

This flexibility makes the platform useful for learning, testing, and experimenting with various programming languages.

3. Queue-Based Task Processing

CloudExecX uses a queue-based architecture powered by BullMQ and Redis. When a user submits code, the execution request is placed in a queue. Worker services then process these tasks asynchronously, ensuring efficient workload management and preventing server overload during high traffic.

4. Auto-Scaling Worker Simulation

The platform demonstrates an auto-scaling model by allowing multiple worker processes to handle queued tasks. This architecture enables the system to simulate scalable execution environments similar to modern cloud-based compute services.

5. Real-Time Execution Output

Users receive execution results directly after the code runs. The platform captures standard output and error streams from the Docker container and sends them back to the user interface, providing immediate feedback.

6. Resource Control and Limits

To maintain fair resource usage and system stability, CloudExecX enforces strict execution limits such as:

- Memory limit (128MB)
- CPU usage restriction (50%)
- Execution timeout (5 seconds)

These limits prevent infinite loops, memory-intensive programs, or resource abuse.

7. Modern Web-Based Code Editor

The platform includes a user-friendly interface built with Next.js and Monaco Editor. This provides syntax highlighting, an interactive coding environment, and a smooth user experience for writing and running code directly in the browser.

8. Authentication and User Management

CloudExecX integrates authentication using Clerk, allowing users to securely log in using email or Google accounts. This ensures that code executions are associated with authenticated users.

Resources Used

The development and deployment of CloudExecX required a combination of software tools, cloud services, and development technologies. These resources helped in building a secure, scalable, and efficient code execution platform.

1. Development Technologies

The following programming languages and frameworks were used to develop the core components of the platform:

- **Next.js 14** – Used to build the frontend web application and user interface.
- **Express.js** – Backend framework used to create RESTful APIs for handling code execution requests.
- **TypeScript** – Provides static typing and improved code reliability for backend development.
- **Tailwind CSS** – Used for designing responsive and modern user interface components.
- **ShadCN UI** – Provides reusable UI components to build a consistent interface.
- **Monaco Editor** – Browser-based code editor used to allow users to write and edit code within the platform.

2. Containerization and Execution Environment

To securely run user-submitted code, the platform uses containerization technologies:

- **Docker** – Used to create isolated containers for executing user code safely.
- **Dockerode** – Node.js library used to interact with Docker containers programmatically.

3. Queue and Job Processing

To manage code execution tasks efficiently, a queue-based system is used:

- **BullMQ** – A robust job queue library used to process background tasks.
- **Upstash Redis** – Cloud-hosted Redis service used to store and manage the execution queue.

4. Database

The platform stores execution results and metadata using:

- **MongoDB Atlas** – Cloud-hosted NoSQL database used for storing application data and logs.

5. Authentication

Secure user authentication is handled using:

- **Clerk** – Provides authentication services including email login and Google OAuth integration.

6. Cloud Hosting and Deployment

Different parts of the system are deployed on cloud platforms:

- **Vercel** – Hosts the frontend application.
- **Render** – Hosts the backend API and worker services.
- **Upstash** – Hosts Redis for queue management.
- **MongoDB Atlas** – Hosts the database service.

7. Development Tools

Additional tools used during development include:

- **Node.js (v20+)** – Runtime environment for backend and worker services.
- **Docker Desktop** – Used for building and running containers locally.
- **Git and GitHub** – Version control and project collaboration.
- **Docker Compose** – Used to orchestrate services during local development.

Algorithm & Flowchart

The following algorithm describes the workflow followed by the CloudExecX platform when a user submits code for execution.

Step 1: Start the system.

Step 2: User logs in to the platform using authentication (Clerk).

Step 3: User writes code in the web-based editor and selects the programming language.

Step 4: User submits the code execution request from the frontend interface.

Step 5: The request is sent to the backend API built with Express.js.

Step 6: The backend validates the request and stores the execution job in the queue using Redis and BullMQ.

Step 7: Worker services continuously monitor the queue for new jobs.

Step 8: When a job is detected, the worker retrieves the code and language configuration.

Step 9: The worker creates a Docker container based on the selected language image.

Step 10: The code is executed inside the container with restricted resources (CPU, memory, timeout).

Step 11: The system captures the program output or error messages.

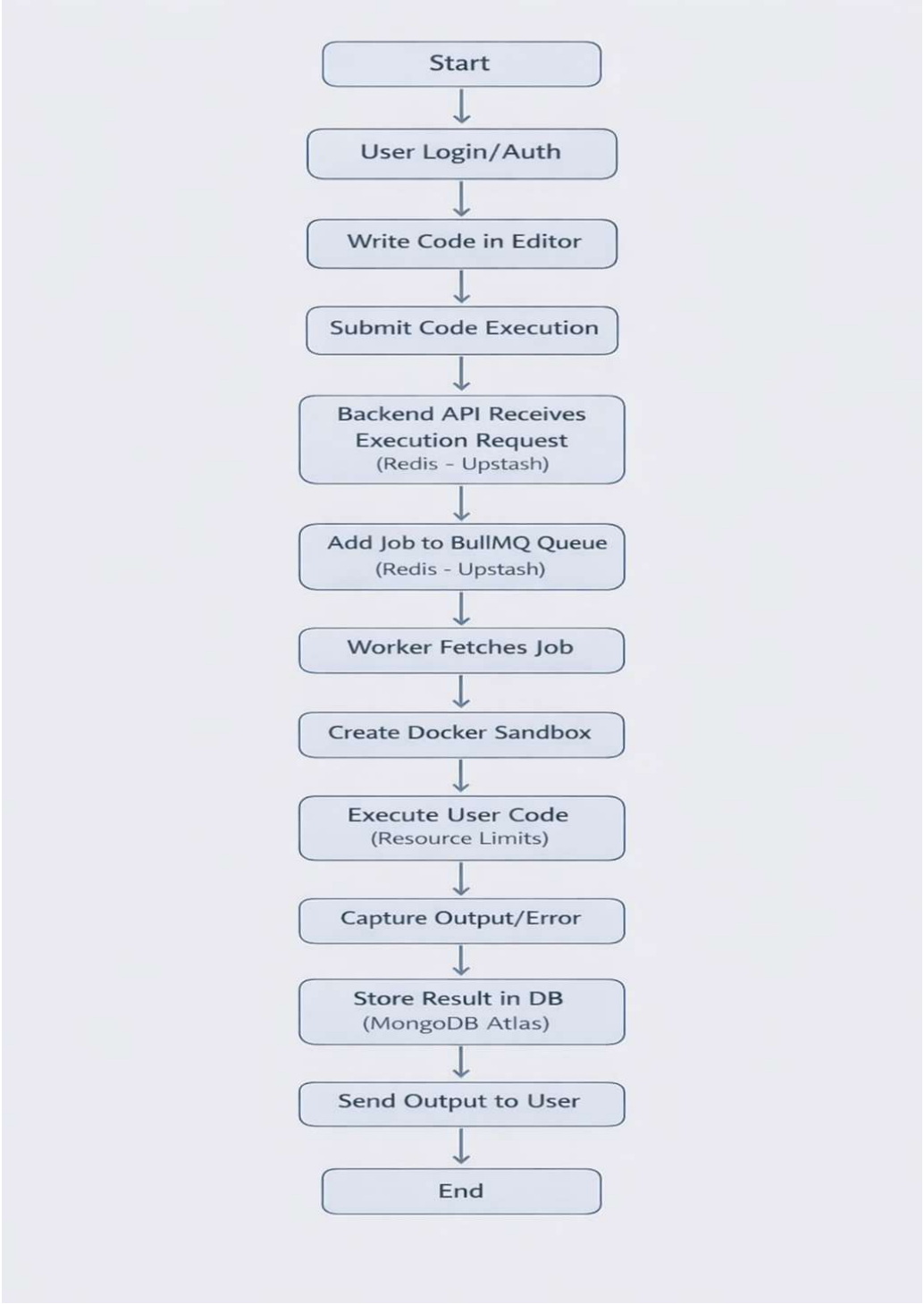
Step 12: Execution results are stored in the database (MongoDB Atlas).

Step 13: The result is sent back to the frontend interface.

Step 14: The user views the output of the executed program.

Step 15: Stop

Flowchart


Start
User Login/Auth
Write Code in Editor
Submit Code Execution
Backend API Receives Execution Request (Redis - Upstash)
Add Job to BullMQ Queue (Redis - Upstash)
Worker Fetches Job
Create Docker Sandbox
Execute User Code (Resource Limits)
Capture Output/Error
Store Result in DB (MongoDB Atlas)
Send Output to User
End

Output

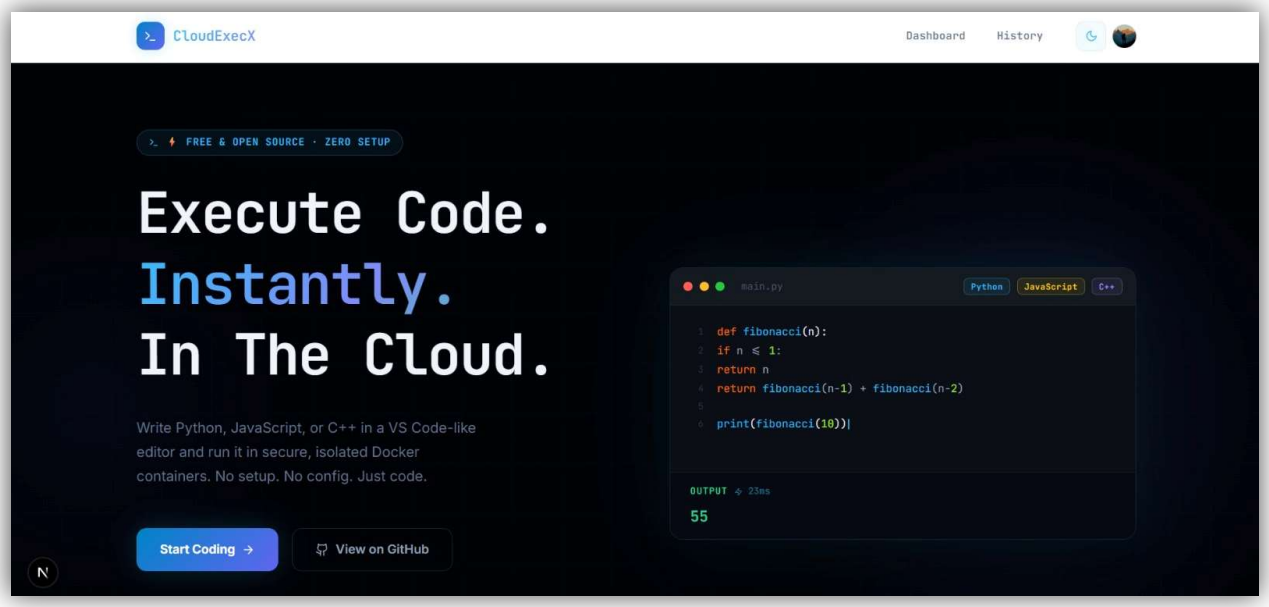


Fig 5.1: User Interface of CloudExecX Platform Enabling Instant Code Execution in Secure Cloud Containers

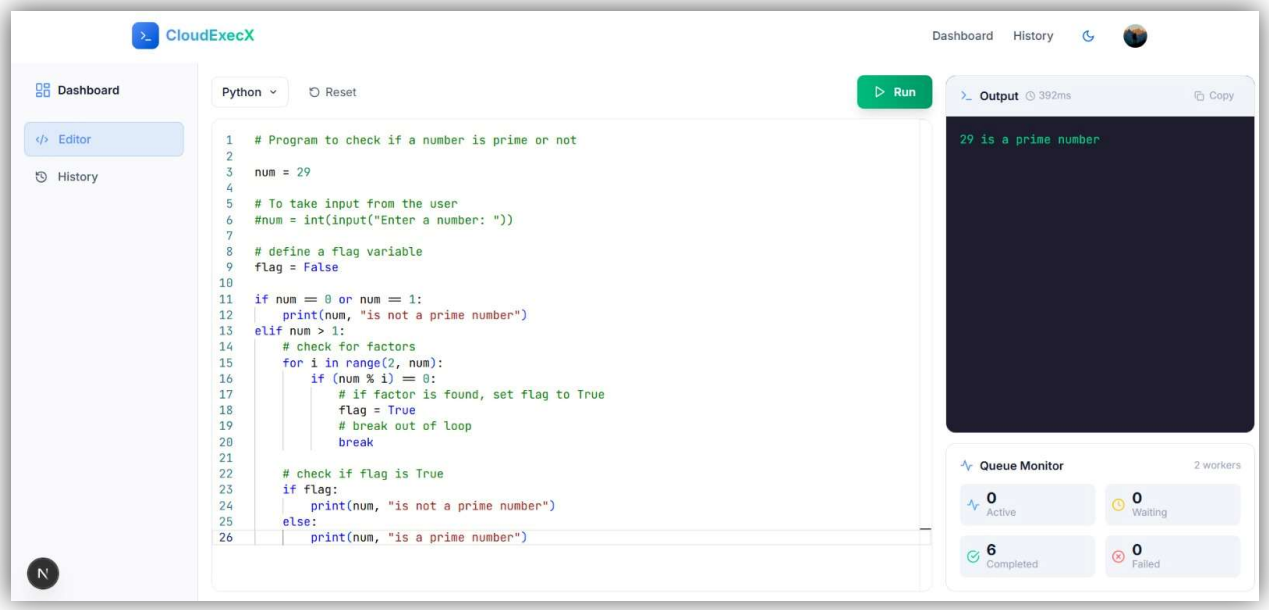


Fig 5.2 : CloudExecX Code Editor Interface Showing Real-Time Code Execution and Output

Conclusion

CloudExecX demonstrates the design and implementation of a secure and scalable cloud-based code execution platform. The system provides users with an interactive environment to write and execute programs in multiple programming languages while ensuring strong isolation and resource control. By utilizing containerization through Docker, each code execution is performed within a sandboxed environment that protects the host system from potential security threats and malicious code.

The platform follows a modern distributed architecture consisting of a frontend interface, backend API, queue-based task management, and worker services responsible for executing code. This architecture enables efficient handling of multiple execution requests while maintaining system stability. The use of a Redis-based queue with BullMQ ensures reliable job processing, while Docker containers enforce strict limits on CPU, memory, and execution time to maintain fair resource utilization.

CloudExecX also highlights how scalable cloud-native systems can be developed using modern web technologies such as Next.js, Express.js, and MongoDB, combined with cloud services like Vercel, Render, and Upstash. An important aspect of the project is its ability to operate entirely on free-tier infrastructure, demonstrating that complex distributed systems can be built and deployed with minimal cost.

Overall, CloudExecX serves as a practical example of applying distributed systems concepts, containerization, and cloud deployment strategies to create a secure code execution platform. The project not only provides a functional development tool but also demonstrates key principles of modern cloud computing, scalability, and system security.